Estado	Finalizado
Comenzado	jueves, 22 de octubre de 2026, 18:24
Completado	jueves, 22 de octubre de 2026, 18:44
Duración	20 mins 14 segs

Pregunta 1

Finalizado

Sin calificar

¿Cómo funciona el proceso básico por el cual un servidor web (como Apache) escucha peticiones entrantes? Explica qué rol juegan las llamadas a la interfaz de *sockets* (como `bind`, `listen` y `accept`) y en qué puerto estandarizado se ejecuta el servicio por defecto.

Un servidor web como Apache usa sockets para recibir peticiones. Los pasos son:

socket(): crea el socket (un punto de comunicación, representado por un descriptor de archivo).
bind(): asocia el socket a una IP y a un puerto. Le dice al sistema operativo: "las conexiones que lleguen a este puerto son para mí".
listen(): pone el socket en modo escucha (pasivo). El kernel empieza a encolar las conexiones entrantes hasta que el servidor las atienda.
accept(): toma una conexión de la cola, completa el handshake TCP y devuelve un nuevo socket dedicado a ese cliente. Por ahí se lee la petición HTTP y se envía la respuesta.

El socket de escucha queda siempre abierto para recibir más clientes; cada accept() crea otro socket solo para esa conexión.

Puerto por defecto: el 80 para HTTP (y el 443 para HTTPS).

- Un servidor web opera como un proceso de fondo (*demonio*) que utiliza la interfaz de **sockets** para comunicarse a través de la capa de transporte (TCP). El flujo de ejecución es el siguiente:
    1. **`socket()`:** El servidor crea un punto de extremo para la comunicación.
    2. **`bind()`:** Asigna la dirección IP local y el puerto al socket. Por defecto, el puerto estandarizado para tráfico HTTP sin cifrar es **el puerto 80** (y el 443 para HTTPS).
    3. **`listen()`:** Le indica al sistema operativo que el socket está preparado para recibir solicitudes de conexión entrantes y define el tamaño de la cola de espera.
    4. **`accept()`:** Bloquea al servidor hasta que llega una petición de un cliente. Cuando el cliente inicia la conexión TCP, `accept()` la completa y crea un nuevo socket dedicado exclusivamente a esa llamada, liberando el socket principal para seguir escuchando en el puerto 80.

Pregunta 2

Finalizado

Sin calificar

En la especificación de HTTP/1.1 se volvió obligatorio el uso del encabezado `Host:`. ¿Qué problema técnico surgió con la proliferación de sitios web que hizo imprescindible esta cabecera para implementar *Virtual Hosting*?

El problema fue el agotamiento de direcciones IPv4 y la necesidad de alojar muchos sitios en un mismo servidor.

Antes (HTTP/1.0): la petición solo incluía la ruta, por ejemplo GET /index.html. El servidor no sabía qué dominio había escrito el cliente, porque el nombre se resolvía por DNS a una IP y después ya no viajaba en la petición. Entonces, para tener varios sitios, cada uno necesitaba su propia dirección IP (virtual hosting basado en IP).

El problema: con la cantidad de sitios web que crecía, no alcanzaban las IPv4 disponibles. Si 100 dominios apuntaban a la misma IP y puerto 80, el servidor no podía distinguir a cuál sitio correspondía cada petición.

La solución (HTTP/1.1): el encabezado Host: indica el dominio solicitado:

GET /index.html HTTP/1.1
Host: www.ejemplo.com

Así, un solo servidor con una IP y un puerto puede alojar muchos sitios. Apache lee el Host: y elige el VirtualHost correcto (virtual hosting basado en nombre, con ServerName).

En las versiones primitivas de HTTP (como HTTP/1.0), la primera línea de la petición solo enviaba la ruta relativa del recurso (por ejemplo, `GET /index.html HTTP/1.0`). La dirección IP de destino se determinaba únicamente en la capa de red (IP).

Con el crecimiento explosivo de la Web y el agotamiento de direcciones IPv4, resultaba inviable y costoso asignar una dirección IP pública dedicada a cada sitio web existente. Para solucionar esto nació el **Virtual Hosting basado en nombres**, que permite alojar múltiples dominios distintos en un mismo servidor con una sola IP. Para que el servidor sepa cuál de todos sus sitios debe entregar, HTTP/1.1 hizo **obligatoria la cabecera `Host:`** en cada petición HTTP (por ejemplo, `Host: mi-proyecto.local`). Así, Apache examina ese campo dentro del texto de la petición y la enruta al `DocumentRoot` correspondiente.

Pregunta 3

Finalizado

Sin calificar

Compara el comportamiento de una conexión HTTP/1.0 (no persistente) frente a HTTP/1.1 (persistente/keep-alive). ¿Qué impacto tiene en la latencia de la red la apertura constante de conexiones TCP?

HTTP/1.0 (no persistente): por cada recurso (HTML, CSS, imagen, JS) se abre una conexión TCP nueva, se hace la petición, se recibe la respuesta y se cierra la conexión.

HTTP/1.1 (persistente, keep-alive): la conexión TCP queda abierta después de la respuesta y se reutiliza para pedir varios recursos, hasta que se cierra por inactividad (timeout) o por indicación del cliente o servidor (Connection: close). En HTTP/1.1 es el comportamiento por defecto.

	HTTP/1.0	HTTP/1.1
Conexiones por página	Una por recurso	Una (o pocas) reutilizada
Handshake TCP	En cada recurso	Una sola vez
Cierre	Después de cada respuesta	Por timeout o `Connection: close`

Impacto en la latencia:

Abrir una conexión TCP requiere el handshake de 3 vías (SYN, SYN-ACK, ACK), que consume 1 RTT (tiempo de ida y vuelta) antes de enviar la primera petición. Si una página tiene 20 recursos, en HTTP/1.0 se paga ese RTT 20 veces.

Además hay otros costos:

- Slow start: cada conexión nueva empieza con una ventana de congestión pequeña, así que al principio transfiere pocos datos por vez y recién después acelera.
- Más carga en el servidor: más sockets abiertos y cerrados, más procesos o hilos ocupados y más recursos del kernel.
- Más tráfico de control: más segmentos SYN, FIN y ACK en la red.

Con keep-alive, el handshake y el slow start se hacen una sola vez y las siguientes peticiones aprovechan la conexión ya "caliente", lo que reduce la latencia total de carga de la página.

- - **HTTP/1.0 (Conexiones No Persistentes):** Abre una conexión TCP separada para solicitar *cada recurso* (el documento HTML, un script, un archivo CSS, una imagen). Al finalizar la transferencia del archivo, la conexión se cierra inmediatamente.
  - **HTTP/1.1 (Conexiones Persistentes / Keep-Alive):** Mantiene la conexión TCP abierta tras enviar la respuesta inicial, permitiendo que las peticiones y respuestas subsiguientes viajen por el mismo canal hasta que se cumpla un tiempo de inactividad (*timeout*) o se cierre explícitamente.

  Impacto en la latencia: En HTTP/1.0, cada recurso individual requiere realizar el proceso de negociación inicial de TCP (*Three-Way Handshake*: SYN, SYN-ACK, ACK), lo que agrega un retardo mínimo de **1 RTT** (Round Trip Time) previo a la transferencia del archivo. En una página web moderna con decenas de archivos adjuntos, las conexiones no persistentes aumentan drásticamente la latencia, saturan el procesamiento del servidor creando y destruyendo sockets, y provocan congestión en la red.

Pregunta 4

Finalizado

Sin calificar

Tanenbaum clasifica los códigos de respuesta del servidor en 5 rangos (1xx a 5xx). Explica la diferencia conceptual entre los rangos **2xx**, **4xx** y **5xx**, citando un ejemplo concreto de cada uno registrado en los archivos de *Log*.

Los códigos de estado se agrupan por su primer dígito, que indica quién tuvo el resultado o el problema.

2xx: Éxito. El servidor recibió, entendió y procesó la petición correctamente.
Ejemplo en el log: 200 OK (se entregó el recurso pedido).

4xx: Error del cliente. La petición es incorrecta o no se puede cumplir por algo que hizo el cliente: URL inexistente, falta de permisos, sintaxis inválida.
Ejemplo en el log: 404 Not Found (el recurso no existe en el servidor). También son comunes 403 Forbidden (sin permisos) y 400 Bad Request.

5xx: Error del servidor. La petición era válida, pero el servidor falló al procesarla (error interno, servicio caído, sobrecarga).
Ejemplo en el log: 500 Internal Server Error (por ejemplo, un script con un error). Otros: 503 Service Unavailable.

Los códigos de estado informan al cliente el resultado de la petición enviada:

- **Rango 2xx (Éxito):** Indica que la solicitud fue recibida, entendida y aceptada correctamente por el servidor.
  - *Ejemplo:* `200 OK`. Aparece en el `access.log` cuando el recurso solicitado (como `index.html`) existe y se entrega sin inconvenientes.
- **Rango 4xx (Error del Cliente):** Indica que la solicitud contiene sintaxis incorrecta o no puede transmitirse porque el error se originó del lado del cliente.
  - *Ejemplo:* `404 Not Found`. Aparece en el `access.log` y `error.log` cuando el cliente solicita un archivo o ruta que no existe dentro del directorio raíz asignado en Apache.
- **Rango 5xx (Error del Servidor):** Indica que el servidor falló o se encontró con un problema técnico que le impide procesar una solicitud que en apariencia es válida.
  - *Ejemplo:* `500 Internal Server Error`. Aparece en el `error.log` si hay un fallo de sintaxis en la configuración interna del servidor (como un error en el archivo `.htaccess`) o si falla un script del lado del servidor.

Pregunta 5

Finalizado

Sin calificar

Antes de que un navegador envíe una petición HTTP GET a un servidor, debe traducir el nombre de dominio a una dirección IP. Explica el orden jerárquico de resolución de nombres que utiliza el sistema y por qué modificar el archivo `hosts` permite "engañar" al cliente local.

Antes de hacer la petición, el sistema necesita saber a qué IP corresponde el dominio. Lo resuelve en este orden (de lo más local a lo más remoto):

- Caché del navegador: si el navegador ya resolvió ese dominio hace poco, usa la IP guardada.
- Caché del sistema operativo (resolver local): el SO también guarda resoluciones recientes.
- Archivo hosts: archivo de texto con pares IP-nombre que se consulta antes de preguntar a cualquier servidor DNS.
  - Linux/macOS: /etc/hosts
  - Windows: C:\Windows\System32\drivers\etc\hosts
- Servidor DNS recursivo: el que tiene configurado el equipo (el del router o el del ISP, o uno público como 8.8.8.8). Si no tiene la respuesta en caché, consulta la jerarquía DNS: servidores raíz → servidores TLD (.com, .ar) → servidor autoritativo del dominio, que devuelve la IP final.

En Linux, el orden entre hosts y DNS lo define el archivo /etc/nsswitch.conf, en la línea hosts: files dns (primero files, o sea /etc/hosts, y después dns).

¿Por qué modificar hosts "engaña" al cliente local?

Porque hosts se consulta antes que el DNS. Si agregás esta línea:

127.0.0.1 www.ejemplo.com

cuando el navegador pida www.ejemplo.com, el sistema encuentra la respuesta en hosts y nunca consulta al DNS. El navegador envía el GET a 127.0.0.1 creyendo que esa es la IP real del sitio.

Hay dos cosas importantes:

- El cambio solo afecta a esa máquina. El resto del mundo sigue resolviendo con el DNS normal.
- Se usa mucho para probar un Virtual Host en Apache sin tener un dominio real: apuntás un nombre inventado a 127.0.0.1, y el navegador envía ese nombre en el encabezado Host:, con lo cual Apache elige el VirtualHost correcto.

Cuando un usuario ingresa una URL (como `[http://mi-proyecto.local](http://mi-proyecto.local)`), la aplicación consulta a la biblioteca de resolución de nombres del sistema operativo (*resolver*) para obtener la dirección IP correspondiente. La jerarquía de consulta sigue este orden estricto:

1. **Caché del navegador y del sistema operativo:** Verifica si la dirección IP se tradujo recientemente.
2. **Archivo local de mapeo (`hosts`):** El sistema operativo lee el archivo físico local (en Windows: `C:\Windows\System32\drivers\etc\hosts`). Si encuentra una coincidencia explicita de un dominio asociado a una IP (por ejemplo: `127.0.0.1 mi-proyecto.local`), **detiene la búsqueda inmediatamente** y devuelve esa IP.
3. **Consulta al Servidor DNS:** Solo si la entrada no se encuentra en el archivo `hosts`, el sistema envía una petición recursiva a través de la red al servidor DNS configurado (usando UDP en el puerto 53).

¿Por qué permite "engañar" al cliente? Al anteponer la consulta al archivo `hosts` sobre la consulta al sistema DNS global, el sistema operativo le responde al navegador que la IP de ese dominio es el entorno de prueba local (`127.0.0.1` / *loopback*), evitando que la petición salga a la red de Internet.